```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>QR Control Timer & Analytics</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://unpkg.com/lucide@latest"></script>
  <script src="https://cdn.jsdelivr.net/npm/jsqr@1.4.0/dist/jsQR.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/qrcode-generator@1.4.4/qrcode.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <script src="https://www.gstatic.com/firebasejs/10.7.1/firebase-app-compat.js"></script>
  <script
src="https://www.gstatic.com/firebasejs/10.7.1/firebase-firestore-compat.js"></script>
</head>
<body class="bg-rose-50/50 text-slate-800 min-h-screen font-sans antialiased pb-12">

  <header class="border-b border-rose-100 bg-white/90 backdrop-blur sticky top-0 z-20 px-4
py-3 shadow-sm">
    <div class="max-w-xl mx-auto flex justify-between items-center">
      <div class="flex items-center gap-2">
        <div class="p-2 bg-rose-500 rounded-xl shadow-md shadow-rose-200">
          <i data-lucide="timer" class="w-5 h-5 text-white"></i>
        </div>
        <div>
          <h1 class="font-bold text-base tracking-wide text-slate-800">QR Control Timer</h1>
          <div id="currentRoomLabel" class="text-[10px] text-rose-500 font-medium">ルーム:
default-room</div>
        </div>
      </div>
      <div class="flex items-center gap-2">
        <button id="openSyncModalBtn" class="text-xs bg-rose-50 border border-rose-200
hover:bg-rose-100 px-2.5 py-1.5 rounded-lg text-rose-700 font-medium flex items-center
gap-1 transition">
          <i data-lucide="wifi" class="w-3.5 h-3.5"></i> 同期・招待
        </button>
        <div id="statusBadge" class="flex items-center gap-1.5 px-3 py-1 rounded-full text-xs
font-medium bg-slate-100 text-slate-500 border border-slate-200">
          <span id="statusDot" class="w-2 h-2 rounded-full bg-slate-400"></span>
          <span id="statusText">待機中</span>
        </div>
      </div>
    </div>
  </header>

  <main class="max-w-xl mx-auto px-4 pt-6 space-y-6">
```

```
   <section class="bg-white border border-rose-100 rounded-3xl p-6 text-center shadow-xl
shadow-rose-950/5 relative overflow-hidden">
     <div id="activeGlow" class="absolute inset-0 bg-rose-500/5 opacity-0 transition-opacity
duration-500 pointer-events-none"></div>
     <div class="text-xs font-semibold text-slate-400 tracking-wider uppercase
mb-1">Elapsed Time</div>
     <div id="timerDisplay" class="font-mono text-5xl sm:text-6xl font-extrabold tracking-tight
text-slate-800 my-2 select-none">
       00:00:00<span class="text-2xl sm:text-3xl text-rose-500">.00</span>
     </div>

     <div class="grid grid-cols-3 gap-2 mt-6 pt-5 border-t border-slate-100">
       <div class="bg-rose-50/40 p-2.5 rounded-2xl border border-rose-100/60">
         <div class="text-[10px] text-slate-500 font-medium">本日</div>
         <div id="sumToday" class="text-xs sm:text-sm font-bold text-rose-600
mt-0.5">00:00:00</div>
       </div>
       <div class="bg-rose-50/40 p-2.5 rounded-2xl border border-rose-100/60">
         <div class="text-[10px] text-slate-500 font-medium">今週</div>
         <div id="sumWeek" class="text-xs sm:text-sm font-bold text-rose-600
mt-0.5">00:00:00</div>
       </div>
       <div class="bg-rose-50/40 p-2.5 rounded-2xl border border-rose-100/60">
         <div class="text-[10px] text-slate-500 font-medium">今月</div>
         <div id="sumMonth" class="text-xs sm:text-sm font-bold text-rose-600
mt-0.5">00:00:00</div>
       </div>
     </div>
   </section>

   <section class="bg-white border border-rose-100 rounded-3xl p-5 shadow-xl
shadow-rose-950/5 space-y-4">
     <div class="flex justify-between items-center">
       <h2 class="text-sm font-bold text-slate-700 flex items-center gap-2">
         <i data-lucide="camera" class="w-4 h-4 text-rose-500"></i>
         QRスキャン（かざしてON/OFF & ルーム参加）
       </h2>
       <button id="toggleCameraBtn" class="text-xs bg-rose-500 hover:bg-rose-600 px-3.5
py-1.5 rounded-xl text-white font-medium transition shadow-md shadow-rose-200">
         カメラ起動
       </button>
     </div>

     <div class="relative aspect-video bg-slate-900 rounded-2xl overflow-hidden border
border-slate-200 flex items-center justify-center">
       <video id="video" class="w-full h-full object-cover hidden" playsinline muted></video>
       <canvas id="canvas" class="hidden"></canvas>
       <div id="cameraPlaceholder" class="text-center p-4 text-slate-400">
```

```
        <i data-lucide="qr-code" class="w-10 h-10 mx-auto mb-2 opacity-40
text-rose-400"></i>
        <p class="text-xs">「カメラ起動」ボタンを押してスキャンを開始してください</p>
       </div>
       <div id="scanOverlay" class="absolute inset-0 border-4 border-rose-500
bg-rose-500/20 opacity-0 transition-opacity pointer-events-none flex items-center
justify-center">
        <span id="scanOverlayText" class="bg-rose-600 text-white text-xs px-3 py-1.5
rounded-full font-bold shadow-md">QRコードを検知！</span>
       </div>
      </div>

      <div class="pt-2 border-t border-slate-100 flex items-center justify-between">
       <div>
        <div class="text-xs font-bold text-slate-700">タイマー制御用QRコード</div>
        <div class="text-[11px] text-slate-500">このQRコードをカメラにかざしてON/OFFを切り
替えます</div>
       </div>
       <button id="showQrBtn" class="bg-rose-50 text-rose-700 border border-rose-200
hover:bg-rose-100 px-3 py-1.5 rounded-xl text-xs font-medium flex items-center gap-1.5
transition">
        <i data-lucide="qr-code" class="w-3.5 h-3.5"></i>
        QRを表示
       </button>
      </div>
    </section>

    <section class="bg-white border border-rose-100 rounded-3xl p-5 shadow-xl
shadow-rose-950/5 space-y-4">
      <div class="flex justify-between items-center">
       <h2 class="text-sm font-bold text-slate-700 flex items-center gap-2">
        <i data-lucide="bar-chart-3" class="w-4 h-4 text-rose-500"></i>
        稼働時間の統計グラフ
       </h2>
       <div class="flex bg-rose-50/60 p-0.5 rounded-xl text-xs font-medium border
border-rose-100">
        <button id="btnGranDaily" class="px-2.5 py-1 rounded-lg bg-white text-rose-600
shadow-sm">日別</button>
        <button id="btnGranWeekly" class="px-2.5 py-1 rounded-lg text-slate-500
hover:text-slate-700">週別</button>
        <button id="btnGranMonthly" class="px-2.5 py-1 rounded-lg text-slate-500
hover:text-slate-700">月別</button>
       </div>
      </div>

      <div class="relative h-56 w-full">
       <canvas id="statsChart"></canvas>
      </div>
```

```
    </section>

    <section class="bg-white border border-rose-100 rounded-3xl p-5 shadow-xl
shadow-rose-950/5 space-y-3">
      <div class="flex justify-between items-center">
        <h2 class="text-sm font-bold text-slate-700 flex items-center gap-2">
          <i data-lucide="history" class="w-4 h-4 text-rose-500"></i>
          記録ログ・作業履歴
        </h2>
        <div class="flex gap-2">
          <button id="exportCsvBtn" class="text-xs bg-slate-50 hover:bg-slate-100 border
border-slate-200 px-2.5 py-1 rounded-lg text-slate-700 transition flex items-center gap-1">
            <i data-lucide="download" class="w-3 h-3"></i> CSV
          </button>
          <button id="clearLogsBtn" class="text-xs bg-red-50 border border-red-200
hover:bg-red-100 px-2.5 py-1 rounded-lg text-red-600 transition">
            消去
          </button>
        </div>
      </div>

      <div class="max-h-60 overflow-y-auto space-y-2 pr-1" id="logsList">
        <div class="text-center py-6 text-xs text-slate-400">まだ記録はありません</div>
      </div>
    </section>
  </main>

  <div id="memoModal" class="fixed inset-0 bg-slate-900/60 backdrop-blur-sm z-50 flex
items-center justify-center p-4 hidden">
    <div class="bg-white border border-rose-100 rounded-3xl max-w-sm w-full p-6 space-y-4
shadow-2xl">
      <div class="flex items-center gap-2 text-rose-500">
        <i data-lucide="edit-3" class="w-5 h-5"></i>
        <h3 class="font-bold text-slate-800">作業内容の記録</h3>
      </div>
      <p class="text-xs text-slate-500">計測が停止しました。行った作業内容を入力してくださ
い。</p>

      <div class="space-y-3">
        <div>
          <label class="text-[11px] font-semibold text-slate-600 block mb-1">カテゴリー</label>
          <select id="categorySelect" class="w-full bg-slate-50 border border-slate-200
rounded-xl px-3 py-2 text-xs text-slate-800 focus:outline-none focus:border-rose-500">
            <option value="勉強">勉強</option>
            <option value="読書">読書</option>
            <option value="開発・プログラミング">開発・プログラミング</option>
            <option value="楽器・音楽">楽器・音楽</option>
            <option value="運動・トレーニング">運動・トレーニング</option>
```

```
          <option value="その他">その他</option>
        </select>
      </div>
      <div>
        <label for="memoInput" class="text-[11px] font-semibold text-slate-600 block mb-1">作業・活動詳細</label>
        <input type="text" id="memoInput" placeholder="例: 数学III 積分問題、英語単語" class="w-full bg-slate-50 border border-slate-200 rounded-xl px-3 py-2 text-sm text-slate-800 focus:outline-none focus:border-rose-500">
      </div>
    </div>

    <div class="flex gap-2 pt-2">
      <button id="skipMemoBtn" class="flex-1 bg-slate-100 hover:bg-slate-200 py-2.5 rounded-xl text-xs font-medium text-slate-600 transition">
        スキップ
      </button>
      <button id="saveMemoBtn" class="flex-1 bg-rose-500 hover:bg-rose-600 py-2.5 rounded-xl text-xs font-bold text-white transition shadow-md shadow-rose-200">
        保存する
      </button>
    </div>
  </div>
</div>

<div id="qrModal" class="fixed inset-0 bg-slate-900/60 backdrop-blur-sm z-50 flex items-center justify-center p-4 hidden">
  <div class="bg-white border border-rose-100 rounded-3xl max-w-xs w-full p-6 text-center space-y-4 shadow-2xl">
    <h3 class="font-bold text-slate-800 text-sm">タイマー制御用QRコード</h3>
    <div id="qrContainer" class="bg-slate-50 p-4 rounded-2xl border border-rose-100 inline-block"></div>
    <p class="text-[11px] text-slate-500">この画面を保存または別端末で表示し、カメラにかざして操作します。</p>
    <button id="closeQrBtn" class="w-full bg-slate-100 hover:bg-slate-200 py-2.5 rounded-xl text-xs font-medium text-slate-700 transition">
      閉じる
    </button>
  </div>
</div>

<div id="syncModal" class="fixed inset-0 bg-slate-900/60 backdrop-blur-sm z-50 flex items-center justify-center p-4 hidden">
  <div class="bg-white border border-rose-100 rounded-3xl max-w-md w-full p-6 space-y-4 shadow-2xl">
    <div class="flex justify-between items-center">
      <h3 class="font-bold text-slate-800 text-sm flex items-center gap-1.5">
        <i data-lucide="wifi" class="w-4 h-4 text-rose-500"></i>
```

```
        複数端末 同期 & ルーム招待
      </h3>
      <button id="closeSyncModalBtn" class="text-slate-400 hover:text-slate-600">
        <i data-lucide="x" class="w-4 h-4"></i>
      </button>
    </div>

    <div class="space-y-3 text-xs">
      <div>
        <label class="block font-semibold text-slate-600 mb-1">ルームID（部屋番号）</label>
        <input type="text" id="roomIdInput" placeholder="例: my-study-room-123"
class="w-full bg-slate-50 border border-slate-200 rounded-xl px-3 py-2 text-slate-800
font-mono focus:outline-none focus:border-rose-500">
      </div>

      <div class="p-3 bg-rose-50/40 rounded-2xl border border-rose-100 space-y-2">
        <label class="block font-semibold text-slate-700">Firebase Firestore APIキー設定
</label>
        <textarea id="firebaseConfigInput" rows="3" placeholder='{"apiKey": "...",
"authDomain": "...", "projectId": "..."}' class="w-full bg-white border border-slate-200
rounded-xl p-2 font-mono text-[10px] text-slate-700 focus:outline-none
focus:border-rose-500"></textarea>
        <p class="text-[10px] text-slate-400">※未入力の場合はローカル環境で動作します。
</p>
      </div>

      <div class="text-center pt-2">
        <div class="text-[11px] font-bold text-slate-700 mb-1">ルーム招待用QRコード</div>
        <div id="roomShareQrContainer" class="bg-slate-50 p-3 rounded-2xl border
border-rose-100 inline-block my-1"></div>
        <p class="text-[10px] text-slate-500">他の端末のカメラでこのQRを読み取ると、一発で
同じ部屋に参加できます！</p>
      </div>
    </div>

    <div class="flex gap-2 pt-2">
      <button id="saveSyncBtn" class="w-full bg-rose-500 hover:bg-rose-600 py-2.5
rounded-xl text-xs font-bold text-white transition shadow-md shadow-rose-200">
        同期設定を保存して接続
      </button>
    </div>
  </div>
 </div>

 <script>
  let TARGET_QR_DATA = "TIMER_TOGGLE_DEFAULT";
  let ROOM_QR_DATA = "TIMER_ROOM_DEFAULT";
  let isRunning = false;
```

```
let startTime = 0;
let elapsedTime = 0;
let timerInterval = null;
let isScanning = false;
let isQrPresent = false;
let pendingLogEntry = null;
let audioCtx = null;
let chartInstance = null;
let currentGranularity = 'daily';

let db = null;
let currentRoomId = "default-room";
let unsubscribeFirestore = null;

const timerDisplay = document.getElementById('timerDisplay');
const activeGlow = document.getElementById('activeGlow');
const statusBadge = document.getElementById('statusBadge');
const statusDot = document.getElementById('statusDot');
const statusText = document.getElementById('statusText');
const toggleCameraBtn = document.getElementById('toggleCameraBtn');
const video = document.getElementById('video');
const canvas = document.getElementById('canvas');
const cameraPlaceholder = document.getElementById('cameraPlaceholder');
const scanOverlay = document.getElementById('scanOverlay');
const scanOverlayText = document.getElementById('scanOverlayText');
const logsList = document.getElementById('logsList');
const memoModal = document.getElementById('memoModal');
const memoInput = document.getElementById('memoInput');
const categorySelect = document.getElementById('categorySelect');
const qrModal = document.getElementById('qrModal');
const syncModal = document.getElementById('syncModal');
const currentRoomLabel = document.getElementById('currentRoomLabel');

document.addEventListener('DOMContentLoaded', () => {
  if (window.lucide) lucide.createIcons();

  const urlParams = new URLSearchParams(window.location.search);
  if (urlParams.has('room')) {
    currentRoomId = urlParams.get('room');
  } else {
    currentRoomId = localStorage.getItem('qr_timer_room_id') || "default-room";
  }
  document.getElementById('roomIdInput').value = currentRoomId;
  updateRoomIdentifiers();

  initFirebaseFromStorage();
  loadLogsAndRender();
  generateControlQr();
```

```
  generateRoomShareQr();
  initChart();
});

function updateRoomIdentifiers() {
  TARGET_QR_DATA = "TIMER_TOGGLE_" + currentRoomId;
  ROOM_QR_DATA = "TIMER_ROOM_" + currentRoomId;
  currentRoomLabel.textContent = `ルーム: ${currentRoomId}`;
}

function initAudio() {
  if (!audioCtx) {
    const AudioContextClass = window.AudioContext || window.webkitAudioContext;
    if (AudioContextClass) audioCtx = new AudioContextClass();
  }
  if (audioCtx && audioCtx.state === 'suspended') audioCtx.resume();
}

function toggleTimer(fromRemote = false) {
  if (isRunning) {
    clearInterval(timerInterval);
    timerInterval = null;
    isRunning = false;
    activeGlow.classList.add('opacity-0');

    const now = new Date();
    pendingLogEntry = {
      id: Date.now(),
      timestamp: now.toISOString(),
      duration: elapsedTime,
      category: categorySelect.value || "その他",
      memo: ""
    };

    memoInput.value = "";
    memoModal.classList.remove('hidden');
    setTimeout(() => memoInput.focus(), 100);

  } else {
    startTime = Date.now() - elapsedTime;
    timerInterval = setInterval(updateTimer, 10);
    isRunning = true;
    activeGlow.classList.remove('opacity-0');
  }

  updateStatusBadge();

  if (!fromRemote && db) {
```

```
      db.collection("rooms").doc(currentRoomId).set({
        isRunning: isRunning,
        startTime: startTime,
        elapsedTime: elapsedTime,
        updatedAt: firebase.firestore.FieldValue.serverTimestamp()
      }, { merge: true });
    }
  }

  function updateTimer() {
    elapsedTime = Date.now() - startTime;
    formatDisplay(elapsedTime);
  }

  function formatDisplay(ms) {
    const totalSec = Math.floor(ms / 1000);
    const hours = Math.floor(totalSec / 3600);
    const minutes = Math.floor((totalSec % 3600) / 60);
    const seconds = totalSec % 60;
    const centiSec = Math.floor((ms % 1000) / 10);

    const hStr = String(hours).padStart(2, '0');
    const mStr = String(minutes).padStart(2, '0');
    const sStr = String(seconds).padStart(2, '0');
    const csStr = String(centiSec).padStart(2, '0');

    timerDisplay.innerHTML = `${hStr}:${mStr}:${sStr}<span class="text-2xl sm:text-3xl
text-rose-500">.${csStr}</span>`;
  }

  function formatTimeSimple(ms) {
    const totalSec = Math.floor(ms / 1000);
    const hours = Math.floor(totalSec / 3600);
    const minutes = Math.floor((totalSec % 3600) / 60);
    const seconds = totalSec % 60;
    return
`${String(hours).padStart(2,'0')}:${String(minutes).padStart(2,'0')}:${String(seconds).padStart
(2,'0')}`;
  }

  function updateStatusBadge() {
    if (isRunning) {
      statusBadge.className = "flex items-center gap-1.5 px-3 py-1 rounded-full text-xs
font-medium bg-rose-50 text-rose-600 border border-rose-200";
      statusDot.className = "w-2 h-2 rounded-full bg-rose-500 animate-pulse";
      statusText.textContent = "計測中";
    } else {
```

```
      statusBadge.className = "flex items-center gap-1.5 px-3 py-1 rounded-full text-xs
font-medium bg-slate-100 text-slate-500 border border-slate-200";
      statusDot.className = "w-2 h-2 rounded-full bg-slate-400";
      statusText.textContent = "待機中";
    }
  }

  document.getElementById('saveMemoBtn').addEventListener('click', () => {
    if (pendingLogEntry) {
      pendingLogEntry.category = categorySelect.value || "その他";
      pendingLogEntry.memo = memoInput.value.trim() || "作業メモなし";
      saveLog(pendingLogEntry);
    }
    closeMemoModal();
  });

  document.getElementById('skipMemoBtn').addEventListener('click', () => {
    if (pendingLogEntry) {
      pendingLogEntry.category = categorySelect.value || "その他";
      pendingLogEntry.memo = "作業メモなし";
      saveLog(pendingLogEntry);
    }
    closeMemoModal();
  });

  function closeMemoModal() {
    memoModal.classList.add('hidden');
    pendingLogEntry = null;
    elapsedTime = 0;
    formatDisplay(0);
  }

  function saveLog(entry) {
    try {
      const logs = JSON.parse(localStorage.getItem('qr_timer_logs_' + currentRoomId) || '[]');
      logs.unshift(entry);
      localStorage.setItem('qr_timer_logs_' + currentRoomId, JSON.stringify(logs));
      loadLogsAndRender();

      if (db) {

db.collection("rooms").doc(currentRoomId).collection("logs").doc(String(entry.id)).set(entry);
      }
    } catch (e) {}
  }

  function loadLogsAndRender() {
    let logs = [];
```

```
    try {
      logs = JSON.parse(localStorage.getItem('qr_timer_logs_' + currentRoomId) || '[]');
    } catch (e) { logs = []; }
    renderLogsList(logs);
    calculateAggregates(logs);
    updateChart(logs);
  }

  function renderLogsList(logs) {
    if (logs.length === 0) {
      logsList.innerHTML = `<div class="text-center py-6 text-xs text-slate-400">まだ記録はあ
りません</div>`;
      return;
    }

    logsList.innerHTML = logs.map(log => {
      const d = new Date(log.timestamp);
      const timeStr = `${d.getMonth()+1}/${d.getDate()}
${String(d.getHours()).padStart(2,'0')}:${String(d.getMinutes()).padStart(2,'0')}`;
      return `
        <div class="bg-rose-50/30 border border-rose-100/60 rounded-2xl p-3 flex
justify-between items-center text-xs shadow-sm">
          <div class="space-y-0.5">
            <div class="flex items-center gap-1.5">
              <span class="px-1.5 py-0.5 bg-rose-100 text-rose-700 text-[10px] font-bold
rounded">${escapeHtml(log.category || 'その他')}</span>
              <span class="font-bold text-slate-800">${escapeHtml(log.memo || '作業メモなし
')}</span>
            </div>
            <div class="text-[10px] text-slate-400">${timeStr}</div>
          </div>
          <div class="font-mono font-bold text-rose-600 text-sm">
            ${formatTimeSimple(log.duration || 0)}
          </div>
        </div>
      `;
    }).join('');
  }

  function calculateAggregates(logs) {
    const now = new Date();
    const todayYear = now.getFullYear();
    const todayMonth = now.getMonth();
    const todayDate = now.getDate();

    const startOfWeek = new Date(now);
    startOfWeek.setDate(now.getDate() - now.getDay());
    startOfWeek.setHours(0,0,0,0);
```

```
    const startOfMonth = new Date(todayYear, todayMonth, 1, 0, 0, 0);

    let todaySum = 0, weekSum = 0, monthSum = 0;

    logs.forEach(log => {
      if (!log.timestamp || !log.duration) return;
      const logDate = new Date(log.timestamp);
      const duration = Number(log.duration) || 0;

      if (logDate.getFullYear() === todayYear && logDate.getMonth() === todayMonth &&
logDate.getDate() === todayDate) {
        todaySum += duration;
      }
      if (logDate >= startOfWeek) weekSum += duration;
      if (logDate >= startOfMonth) monthSum += duration;
    });

    document.getElementById('sumToday').textContent = formatTimeSimple(todaySum);
    document.getElementById('sumWeek').textContent = formatTimeSimple(weekSum);
    document.getElementById('sumMonth').textContent = formatTimeSimple(monthSum);
  }

  function initChart() {
    const ctx = document.getElementById('statsChart').getContext('2d');
    chartInstance = new Chart(ctx, {
      type: 'bar',
      data: { labels: [], datasets: [{ label: '作業時間 (分)', data: [], backgroundColor: 'rgba(244,
63, 94, 0.85)', borderRadius: 6 }] },
      options: {
        responsive: true,
        maintainAspectRatio: false,
        scales: { y: { beginAtZero: true, grid: { color: '#f1f5f9' } }, x: { grid: { display: false } } },
        plugins: { legend: { display: false } }
      }
    });
  }

  function updateChart(logs) {
    if (!chartInstance) return;
    const labels = [];
    const data = [];
    const map = {};

    logs.forEach(l => {
      const d = new Date(l.timestamp);
      let key = `${d.getMonth()+1}/${d.getDate()}`;
      if (currentGranularity === 'monthly') key = `${d.getFullYear()}/${d.getMonth()+1}`;
```

```
      map[key] = (map[key] || 0) + Math.floor((l.duration || 0) / 60000);
    });

    Object.keys(map).slice(0, 7).reverse().forEach(k => {
      labels.push(k);
      data.push(map[k]);
    });

    chartInstance.data.labels = labels;
    chartInstance.data.datasets[0].data = data;
    chartInstance.update();
  }

  document.getElementById('btnGranDaily').addEventListener('click', () =>
setGranularity('daily'));
  document.getElementById('btnGranWeekly').addEventListener('click', () =>
setGranularity('weekly'));
  document.getElementById('btnGranMonthly').addEventListener('click', () =>
setGranularity('monthly'));

  function setGranularity(g) {
    currentGranularity = g;
    ['Daily', 'Weekly', 'Monthly'].forEach(name => {
      const btn = document.getElementById('btnGran' + name);
      if (name.toLowerCase() === g) {
        btn.className = "px-2.5 py-1 rounded-lg bg-white text-rose-600 shadow-sm";
      } else {
        btn.className = "px-2.5 py-1 rounded-lg text-slate-500 hover:text-slate-700";
      }
    });
    loadLogsAndRender();
  }

  toggleCameraBtn.addEventListener('click', () => {
    initAudio();
    if (isScanning) stopCamera(); else startCamera();
  });

  async function startCamera() {
    try {
      const stream = await navigator.mediaDevices.getUserMedia({ video: { facingMode:
"environment" } });
      video.srcObject = stream;
      video.setAttribute("playsinline", "true");
      video.muted = true;
      await video.play();
      isScanning = true;
```

```
        video.classList.remove('hidden');
        cameraPlaceholder.classList.add('hidden');
        toggleCameraBtn.textContent = "カメラ停止";
        toggleCameraBtn.className = "text-xs bg-red-600 hover:bg-red-500 px-3.5 py-1.5
rounded-xl text-white font-medium transition shadow-md";
        requestAnimationFrame(tick);
      } catch (err) { alert("カメラの起動に失敗しました。パーミッションをご確認ください。"); }
    }

    function stopCamera() {
      if (video.srcObject) video.srcObject.getTracks().forEach(t => t.stop());
      isScanning = false;
      video.classList.add('hidden');
      cameraPlaceholder.classList.remove('hidden');
      toggleCameraBtn.textContent = "カメラ起動";
      toggleCameraBtn.className = "text-xs bg-rose-500 hover:bg-rose-600 px-3.5 py-1.5
rounded-xl text-white font-medium transition shadow-md shadow-rose-200";
    }

    let frameCount = 0;
    function tick() {
      if (!isScanning) return;
      if (video.readyState === video.HAVE_ENOUGH_DATA) {
        frameCount++;
        if (frameCount % 5 === 0) {
          const scale = 0.5;
          canvas.width = video.videoWidth * scale;
          canvas.height = video.videoHeight * scale;
          const ctx = canvas.getContext("2d");
          ctx.drawImage(video, 0, 0, canvas.width, canvas.height);
          const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);

          if (window.jsQR) {
            const code = jsQR(imageData.data, imageData.width, imageData.height, {
inversionAttempts: "dontInvert" });
            if (code) {
              if (code.data === TARGET_QR_DATA) {
                if (!isQrPresent) {
                  isQrPresent = true;
                  scanOverlayText.textContent = "タイマー切り替え検知！";
                  triggerFeedback();
                  toggleTimer();
                }
              } else if (code.data.startsWith("TIMER_ROOM_")) {
                const newRoom = code.data.replace("TIMER_ROOM_", "");
                if (newRoom && newRoom !== currentRoomId) {
                  if (!isQrPresent) {
                    isQrPresent = true;
```

```
                scanOverlayText.textContent = `ルーム「${newRoom}」に参加！`;
                triggerFeedback();
                stopCamera();
                switchRoom(newRoom);
              }
            }
          }
        } else { isQrPresent = false; }
      }
    }
  }
  requestAnimationFrame(tick);
}

function switchRoom(newRoom) {
  currentRoomId = newRoom;
  localStorage.setItem('qr_timer_room_id', currentRoomId);
  updateRoomIdentifiers();
  generateControlQr();
  generateRoomShareQr();
  initFirebaseFromStorage();
  loadLogsAndRender();
  alert(`ルーム「${currentRoomId}」に接続しました！`);
}

function triggerFeedback() {
  scanOverlay.classList.remove('opacity-0');
  setTimeout(() => scanOverlay.classList.add('opacity-0'), 800);
  if (audioCtx) {
    try {
      const osc = audioCtx.createOscillator();
      const gain = audioCtx.createGain();
      osc.type = "sine";
      osc.frequency.setValueAtTime(880, audioCtx.currentTime);
      gain.gain.setValueAtTime(0.1, audioCtx.currentTime);
      osc.connect(gain); gain.connect(audioCtx.destination);
      osc.start(); osc.stop(audioCtx.currentTime + 0.15);
    } catch (e) {}
  }
}

function generateControlQr() {
  if (window.qrcode) {
    const qr = qrcode(0, 'M');
    qr.addData(TARGET_QR_DATA);
    qr.make();
    document.getElementById('qrContainer').innerHTML = qr.createImgTag(5);
  }
```

```
  }

  function generateRoomShareQr() {
    if (window.qrcode) {
      const qr = qrcode(0, 'M');
      qr.addData(ROOM_QR_DATA);
      qr.make();
      document.getElementById('roomShareQrContainer').innerHTML = qr.createImgTag(4);
    }
  }

  document.getElementById('showQrBtn').addEventListener('click', () =>
qrModal.classList.remove('hidden'));
  document.getElementById('closeQrBtn').addEventListener('click', () =>
qrModal.classList.add('hidden'));

  document.getElementById('openSyncModalBtn').addEventListener('click', () =>
syncModal.classList.remove('hidden'));
  document.getElementById('closeSyncModalBtn').addEventListener('click', () =>
syncModal.classList.add('hidden'));

  document.getElementById('saveSyncBtn').addEventListener('click', () => {
    const newRoom = document.getElementById('roomIdInput').value.trim() || "default-room";
    const configStr = document.getElementById('firebaseConfigInput').value.trim();

    currentRoomId = newRoom;
    localStorage.setItem('qr_timer_room_id', currentRoomId);
    if (configStr) localStorage.setItem('qr_timer_firebase_config', configStr);

    updateRoomIdentifiers();
    generateControlQr();
    generateRoomShareQr();
    initFirebaseFromStorage();
    loadLogsAndRender();
    syncModal.classList.add('hidden');
  });

  function initFirebaseFromStorage() {
    if (unsubscribeFirestore) {
      unsubscribeFirestore();
      unsubscribeFirestore = null;
    }
    const configStr = localStorage.getItem('qr_timer_firebase_config');
    if (!configStr) return;
    try {
      const config = JSON.parse(configStr);
      if (!firebase.apps.length) firebase.initializeApp(config);
      db = firebase.firestore();
```

```
      unsubscribeFirestore = db.collection("rooms").doc(currentRoomId).onSnapshot(doc =>
{
        if (doc.exists) {
          const data = doc.data();
          if (data.isRunning !== undefined && data.isRunning !== isRunning) {
            toggleTimer(true);
          }
        }
      });
    } catch (e) {}
  }

  document.getElementById('exportCsvBtn').addEventListener('click', () => {
    let logs = [];
    try { logs = JSON.parse(localStorage.getItem('qr_timer_logs_' + currentRoomId) || '[]'); }
catch (e) {}
    if (logs.length === 0) return alert("データがありません");

    let csv = "\uFEFF日時,カテゴリー,作業内容,経過時間(秒)\n";
    logs.forEach(l => {
      csv += `"${l.timestamp}","${l.category||'その他'}","${(l.memo||'').replace(/"/g,
'""')}",${Math.floor((l.duration||0)/1000)}\n`;
    });
    const blob = new Blob([csv], { type: 'text/csv;charset=utf-8;' });
    const link = document.createElement("a");
    link.href = URL.createObjectURL(blob);
    link.download = `timer_logs_${currentRoomId}_${new
Date().toISOString().slice(0,10)}.csv`;
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
  });

  document.getElementById('clearLogsBtn').addEventListener('click', () => {
    if (confirm("すべての計測履歴を消去しますか？")) {
      localStorage.removeItem('qr_timer_logs_' + currentRoomId);
      loadLogsAndRender();
    }
  });

  function escapeHtml(str) {
    if (!str) return '';
    return String(str).replace(/[&<>"']/g, m => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'":
'&#39;' })[m]);
  }
 </script>
</body>
```

</html>